

C05 - Assessment Tasks 2

Error Analysis Task

SET1

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

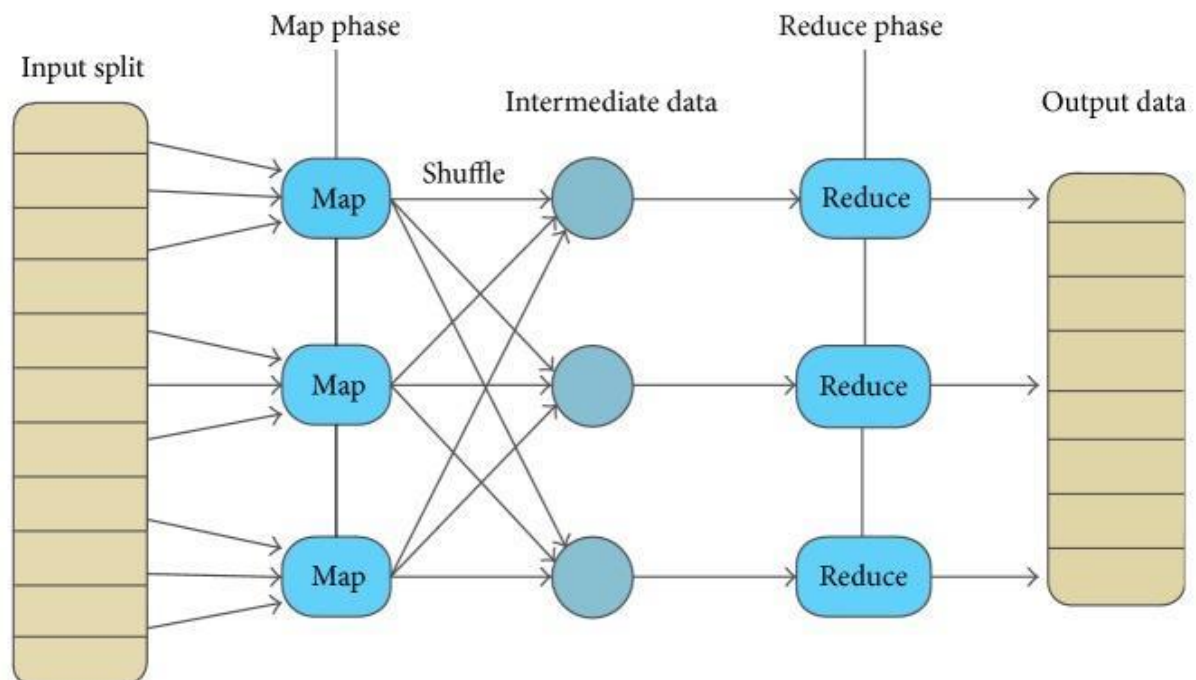
Question 1: Uneven Input Splits and Idle Reducers

Problem Statement

A Hadoop job repeatedly fails because input splits are uneven, causing some mappers to process much larger amounts of data while several reducers remain idle.

Problem Analysis

This problem usually occurs because the data is poorly partitioned. Hadoop automatically creates input splits based on HDFS block size. If files are uneven in size or keys are distributed unevenly, the workload becomes unbalanced.



In this situation:

- Some mappers receive huge input blocks.
- Other mappers finish quickly.

- Reducers receive uneven key distributions.
- Cluster resources remain underutilized.

Likely Design Error

The main design mistakes are:

- Uneven input split generation
- Poor partitioning strategy
- Data skew during shuffle phase
- Incorrect reducer configuration

Causes

Cause	Explanation
Uneven file sizes	Large files create overloaded mappers
Data skew	One reducer receives most records
Poor partitioning	Keys are not distributed evenly
Fixed reducer count	Reducers do not match workload

Architecture Diagram

Corrective Actions

- Increase input parallelism.
- Use CombineFileInputFormat for small files.
- Implement a Custom Partitioner.
- Enable Combiner.
- Adjust reducer count dynamically.
- Compress intermediate data.

Best Practices

- Monitor mapper execution time.
- Balance key distribution.
- Avoid oversized input blocks.
- Tune mapreduce.job.reduces.

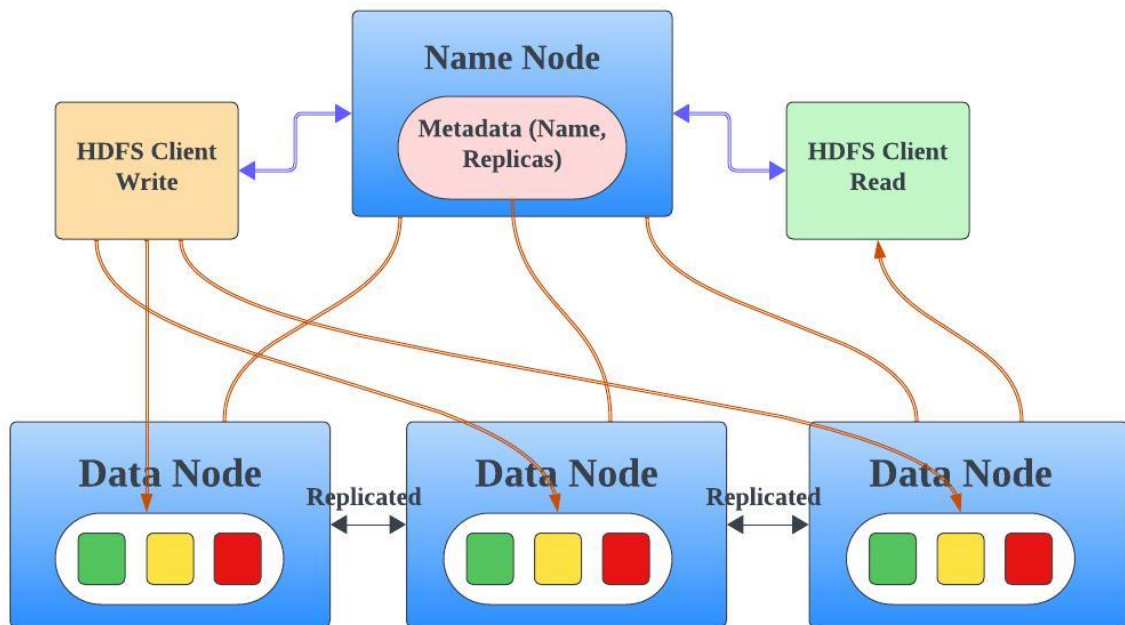
Question 2: Frequent Data Unavailability After One Node Failure

Problem Statement

An HDFS deployment experiences frequent data unavailability immediately after a single DataNode failure.

Problem Analysis

HDFS is designed to tolerate node failures through replication. If data becomes unavailable after only one node fails, replication or metadata management has been configured incorrectly.



Likely Design Error

Possible design mistakes include:

- Replication factor set too low
- Replicas stored on same rack
- NameNode configuration problems
- Missing rack awareness

Root Causes

Cause	Effect
Replication factor = 1	No backup copy
Same-rack replicas	Rack failure loses all copies
NameNode issue	Block locations unavailable
Delayed re-replication	Temporary data loss

Architecture Diagram

Recommended Solutions

- Set replication factor to 3.
- Enable rack-aware replication.
- Deploy Secondary NameNode or HA NameNodes.
- Monitor replication continuously.
- Replace failed nodes immediately.

Best Practices

- Maintain healthy replication.
- Perform regular block reports.
- Enable automatic rebalancing.

Question 3: Apache NiFi Creates Duplicate Records

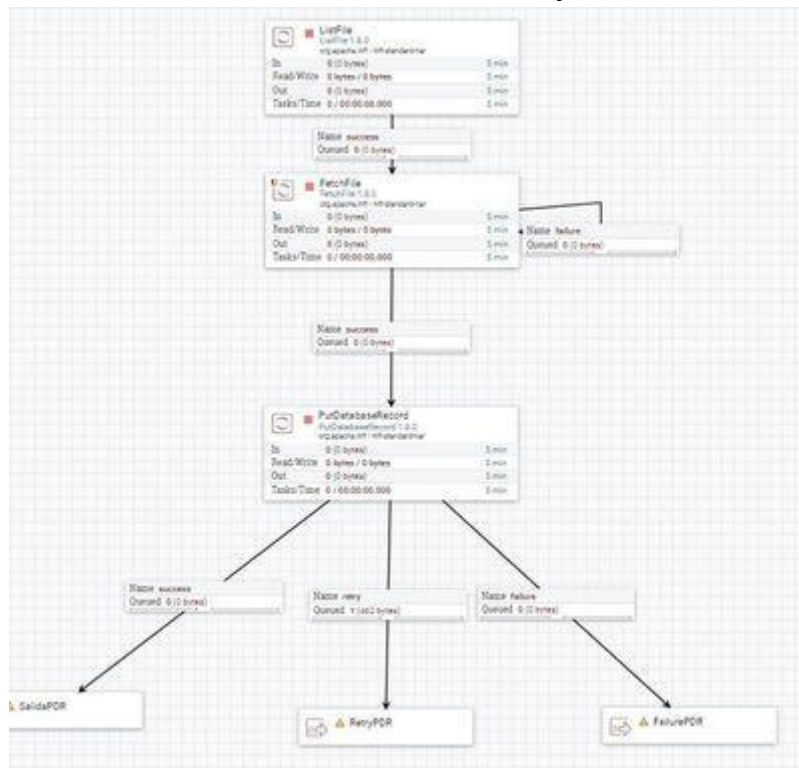
Problem Statement

An ETL workflow using Apache NiFi inserts duplicate records into the analytics database.

Problem Analysis

Duplicate records usually indicate that the ingestion process is not idempotent. When NiFi

retries failed FlowFiles, the destination may receive the same data multiple times.



Likely Causes

Cause	Description
FlowFile retry	Same record processed again
No unique key	Database cannot detect duplicates
At-least-once delivery	Duplicate insertion
Processor restart	Data replay

Architecture Diagram

Recommended Fixes

- Use idempotent processing.
- Generate unique record IDs.
- Enable deduplication processors.
- Use UPSERT instead of INSERT.
- Track processed records.
- Configure retry limits.

Best Practices

- Validate incoming data.
- Use transactional writes.
- Monitor FlowFile queues.

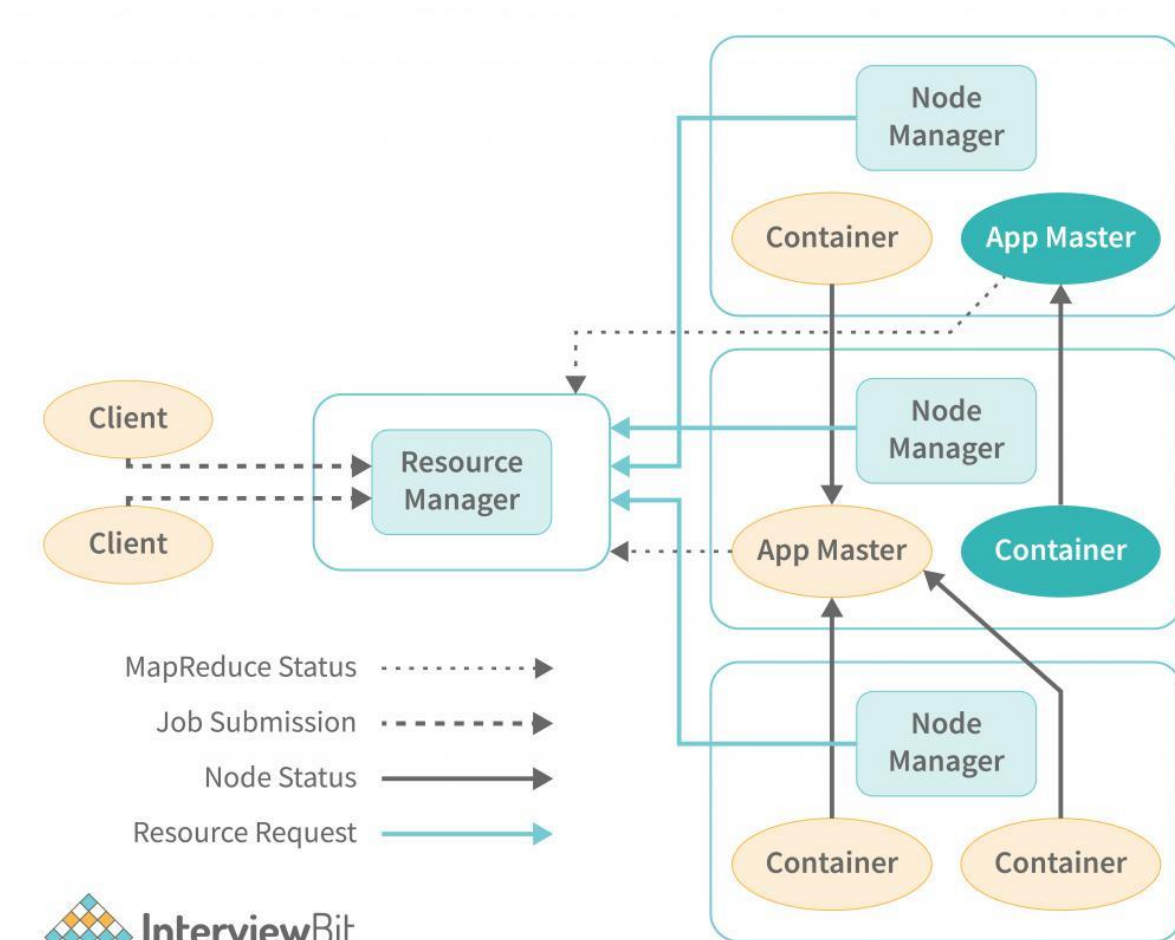
Question 4: YARN Resource Starvation

Problem Statement

A YARN cluster suffers from resource starvation when many users submit jobs simultaneously.

Problem Analysis

Resource starvation happens when the scheduler allocates resources unfairly, causing some applications to wait indefinitely.



Likely Scheduling Error

- Default scheduler poorly configured.
- No queue limits.
- One user consumes excessive resources.
- Containers improperly sized.

Root Causes

Cause	Impact
Large containers	Fewer jobs run
No user limits	Resource monopolization
Poor queue setup	Long waiting times
Memory fragmentation	Idle resources

Architecture Diagram

Better Resource Management

- Enable Capacity Scheduler.
- Use Fair Scheduler.
- Configure queue quotas.
- Set user limits.
- Enable preemption.
- Tune container memory and CPU.

Best Practices

- Separate production and development queues.

- Monitor cluster utilization.
- Prevent resource monopolization.

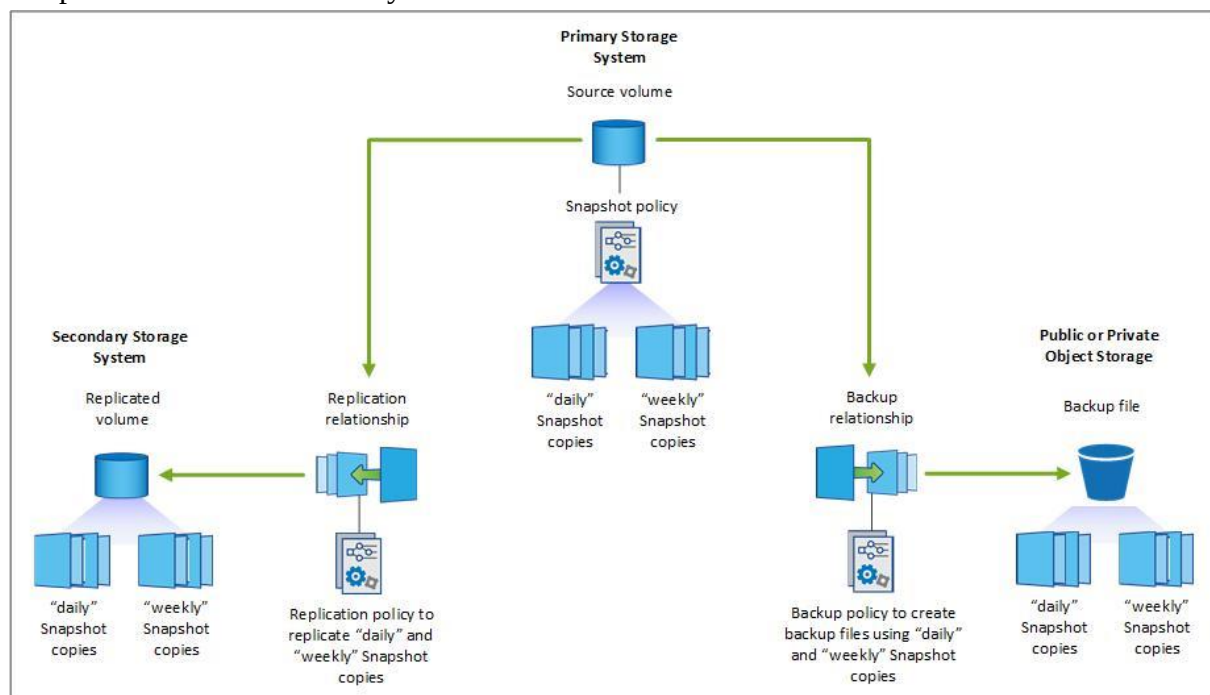
Question 5: Backup Restores Outdated Data

Problem Statement

A distributed storage backup restores outdated data after system recovery.

Problem Analysis

The backup process is not synchronized with current data updates. Replication alone does not provide historical recovery.



Probable Design Flaw

- Old backup snapshots
- No incremental backups
- Replication mistaken for backup
- Poor backup scheduling

Causes

Cause	Effect
Weekly backup only	Data becomes outdated
No snapshot	Latest changes lost
Replication only	Corrupted data copied
No versioning	Cannot restore correct state

Architecture Diagram

Recommended Fixes

- Use HDFS snapshots.
- Perform incremental backups.
- Maintain backup version history.
- Verify backups regularly.
- Separate backup from replication.
- Automate backup validation.

Best Practices

- Follow the 3-2-1 backup rule.
- Test disaster recovery.
- Encrypt backup copies.
- Maintain retention policies.

Overall Comparison Table

Question	Main Component	Design Error	Recommended Fix
Q1	MapReduce	Uneven input splits	Better partitioning
Q2	HDFS	Low replication	Replication factor 3
Q3	Apache NiFi	Duplicate processing	Idempotent ingestion
Q4	YARN	Poor scheduling	Capacity/Fair Scheduler
Q5	Backup	Outdated backups	Snapshots + Incremental Backup

References

1. Apache Hadoop Official Documentation (HDFS, YARN, MapReduce)
2. Apache NiFi Documentation
3. Apache Hive Documentation
4. Apache Spark Documentation
5. Tom White, Hadoop: The Definitive Guide, O'Reilly Media
6. Cloudera Hadoop Performance Tuning Guide